

SMARTDINE – CONTACTLESS DINING EXPERIENCE USING QR CODE

**A project Report submitted of the partial fulfillment of the
requirements for the award of the degree of**

BACHELOR OF COMPUTER APPLICATIONS

By

MUBASHERA S - 31223U09052

NAZEEFA SUMAN K - 31223U09056

RAGAVI P - 31223U09067

SAVI PRIYA K - 31223U09084

SWEATHA S - 31223U09088

UNDER THE GUIDANCE OF

PROF.C.RANJITHA M.Sc.,M.Phil.,



DEPARTMENT OF COMPUTER APPLICATIONS

ISLAMIAH WOMEN'S ARTS AND SCIENCE

COLLEGE(AUTONOMOUS)

(Accredited by NAAC with 'B++' grade)

(Recognized by UGC under section 2(f) and 12(B) of UGC Act 1956 And

Permanently Affiliated with Thiruvalluvar University)

VANIYAMBADI-635 752

DEPARTMENT OF COMPUTER APPLICATION

CERTIFICATE

This is to certify that the project work entitled, “**SMART DINE – QR BASED ORDERING SYSTEM**” is the bonafide record done by **MUBASHERA S (31223U09052), NAZEEFA SUMAN K (31223U09056), RAGAVI P (31223U09067), SAVI PRIYA K (31223U09084), SWETHA S (31223U09088).** in partial fulfillment of the requirement for the award of the degree the **BACHELOR OF COMPUTER APPLICATION** Submitted at Islamiah womens Arts and Science college (Autnomous), Vaniyambadi affiliated to Thiruvalluvar University, is an authentic record of their work carried out during a period from June to October under the guidance of **PROF.C.RANJITHA M.Sc.,M.Phil.,** Assistant professor, Depart of Computer application. The major project viva-voice examination held

On _____

Signature of the Guide

Signature of the H.O.D

Internal Examiner

External Examiner

Signature of the principal

DECLARATION

I hereby declare that the project work entitled "**SMART DINE - QR BASED ORDERING SYSTEM**" submitted to Islamiah Women's Arts Science college (AUTONOMOUS), Vaniyambadi, partial fulfillment of the requirements for the award of the **BACHELOR OF COMPUTER APPLICATION** is a record original independent project work done by me during the period OCT-2025 under the guidance of **PROF.C.RANJITHA.,M.Sc.,M.Phil., and T.ARUNA.,M.C.A.,M.Phil.,SET., HEAD of the Department of Computer Application,** Islamiah Women's Arts Science College (AUTONOMOUS), Vaniyambadi.

PLACE: VANIYAMBADI

DATE:

SIGNATURE OF THE CANDIDATE

ACKNOWLEDGEMENT

First of all, I thank the almighty **Allah** who showered his immense blessings on me, which helped me to complete this project successfully. I also thank my beloved Parents and family members for giving me their love and support for all my endeavors.

I sincerely thank, respected Secretary and general correspondent for the encouragement and for permitting me to do this project.

My special and hearty thanks to Principal **Dr. M. RENU M.B.A(HR)., MBA(FIN)., M.Com., M.Phil., PGDCA., SET., PhD.**, Islamiah Women's Art's and Science College (Autonomous), Vaniyambadi for giving the necessary environment to acquire knowledge and skill.

My sincere and special thanks to project guide **PROF.C.RANJITHA M.Sc.,M.Phil., Prof.T.Aruna M.C.A.,M.Phil.,SET.**, HOD, Department of Computer Application, Islamiah Women's Art's and Science College,Vaniyambadi for her encouragement for the successful completion of this project.

My sincere thanks to Librarian and other Faculties, who deliberately provided their consistent support for completing my project. I thank all of them for their incredible support.

TABLE CONTENT

CHAPTER NO	PARTICULAR	PAGE NO
01	ABSTRACT	1
	INTRODUCTION	3
	EXISTING SYSTEM	3
	PROPOSED SYSTEM	3
02	SOFTWARE SPECIFICATION	8
	HARDWARE SPECIFICATION	8
	SYSTEM SPECIFICATION	8
03	SYSTEM DESIGN	11
	DESIGN METHODOLOGY	12
	DATA FLOW DIAGRAM	16
	CLASS DIAGRAM	17
	SEQUENCE DIAGRAM	18
04	IMPLEMENTATION AND CODING	21
05	SAMPLE OUTPUT	51
06	UNIT TESTING	57
	INTEGRATION TESTING	57
	ACCEPTANCE TESTING	58
07	CONCLUSION	60
08	BIBLIOGRAPHY	61

ABSTRACT

The Smart Dine System is an innovative restaurant management solution designed to digitalize the traditional dining experience. The main objective of this project is to simplify the process of ordering food in restaurants by using QR code-based technology. Each dining table in the restaurant is assigned a unique table name and QR code. When customers scan the QR code using their smartphones, they are directed to the restaurant's online menu interface.

From this interface, customers can conveniently browse the menu, select their preferred dishes, and place orders directly from their table without waiting for a waiter to take the order manually. Once the order is confirmed, it is automatically sent to the kitchen system and waiter's dashboard, where it can be prepared and served efficiently.

CHAPTER-1

INTRODUCTION

1.1 INTRODUCTION

The aim of the project Smart Dine is to digitalize and simplify the restaurant dining experience through the use of QR code technology. In most traditional restaurants, customers depend on waiters to take orders, which often leads to longer waiting times, communication errors, and service delays—especially during peak hours.

The Smart Dine System is designed to overcome these limitations by enabling customers to place orders directly from their table using their smartphones. Each dining table is assigned a unique table name and QR code. When scanned, the QR code redirects the customer to a web-based menu interface, where they can view available dishes, select items, and confirm their order instantly.

The different process to be used for our project is :

1. user Login
2. Admin Login
3. Reservation
4. Application
5. Schemes

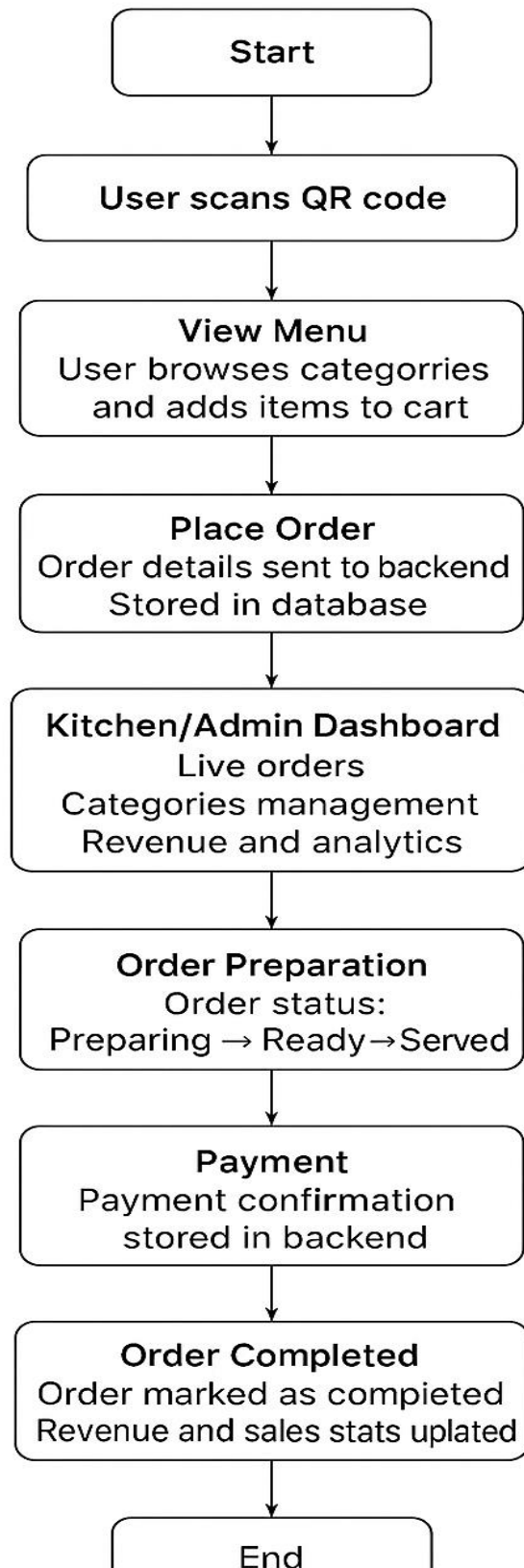
1.2 Existing System:

The existing restaurant order management system relies heavily on manual operations. Customers must wait for a waiter to approach their table, take the order, and pass it to the kitchen staff. This manual process often leads to problems such as:

1.3 Proposed System

The proposed system replaces traditional menu cards and manual order-taking with a digital, contactless solution. Each table is equipped with a unique QR code that customers scan to access the menu, place orders, and make payments—all from their smartphones.

ADMIN PANEL



1. Admin Login

- Secure access to the admin panel.
- Only authorized personnel can manage the system.

2. Admin Panel

- Dashboard or interface where admin can control different aspects of the SmartDine system.

3. Core Modules Managed by Admin:

- **Table Management**

Assign and manage table numbers, occupancy status, QR code links, etc.

- **Menu Management**

Add/edit/delete menu items, categories, prices, and availability.

- **Order Management**

Track customer orders in real time and ensure kitchen coordination.

- **Payment Tracking**

Monitor payment status, history, refunds, and payment gateway integrations.

- **Staff Management**

Add or manage waiter, chef, cleaner, and delivery roles with access controls.

4. Feedback Module

- Collects customer ratings and comments after dining experience.

5. Analyse Ratings

- Admin reviews customer feedback to determine satisfaction or issues.

6. Respond to Feedback

- If necessary, the admin takes action based on negative ratings (e.g., compensations, improvements).
- A feedback loop ensures continuous quality improvement.

CHAPTER-2

SPECIFICATION

2.1 Software Specification:

● **Visual Studio Code:**

Visual Studio Code is a streamlined code editor with support for development operations like debugging, task running, and version control. It aims to provide just the tools a developer needs for a quick code-build-debug cycle and leaves more complex workflows to fuller featured IDEs, such as Visual Studio IDE.

● **My SQL Server:**

MySQL is an Oracle-Based open-source Relational database management system based on Structured Query Language. MySQL runs on virtually all platforms, including Linux, Windows.

MySQL is the RDBMS behind many of the top of the websites in the world and countless corporate and consumer-facing web-based application, including Facebook, Twitter and YouTube.

Since Computers are very good at handling large amounts of data, database management systems play a central role in computing, as standalone utilities, or as parts of other application.

2.2 Hardware Specification:

- ❖ Hard Disk Capacity: more than 100GB.

2.3 SYSTEM SPECIFICATION

HARDWARE CONFIGURATION

- * Processor - Intel i3
- * Speed - 2.5GHz
- * RAM - 2GB (min)
- * Hard Disk - 500 GB
- * Keyboard - Standard Windows Keyboard
- * Mouse - Two or Three Button Mouse

SOFTWARE CONFIGURATION

- * Operating System - Windows 10
- * Coding Language - HTML,CSS,JAVA SCRIPT
- * Data Base - NODEJS,react
- * IDE - VISUAL STUDIO CODE

CHAPTER-3

SYSTEM DESIGN

3.1 System Design:

The System Design Phase is one of the most crucial stages in software development. It translates the requirements gathered during the analysis phase into a blueprint for implementation. The purpose of system design is to define how the system's components — including user interfaces, databases, and process flows — will work together to achieve the project's objectives.

In the Smart Dine System, design focuses on creating a seamless interaction between the customer, waiter, and admin through digital communication. The system ensures that when a customer scans the QR code on a table, the appropriate table name and menu are displayed, and the order details are automatically sent to the kitchen and waiter panel.

The design process involves defining modules such as:

- Customer Interface design (Menu and Order Pages)
- Admin Management System
- Database Design for orders, tables, and menu items

The system design helps visualize how data flows between different modules, ensuring that the ordering process is fast, accurate, and efficient.

Purpose of System Design:

- Defines **how components interact** (customer, waiter, admin)
- Ensures **accurate, fast data flow** from QR scan to kitchen
- Lays foundation for a **seamless digital dining experience**

Functional Modules:

1. QR Code Module – Generates and identifies unique table QR codes.
2. Customer Interface – Displays menu and order form for customers.
3. Order Management Module – Sends order details to the kitchen and waiter panel.
4. Admin Module – Allows the admin to manage tables, menu items, and orders.
5. Database Module – Stores and retrieves data efficiently for all operations.

3.2 DESIGN METHODOLOGY

The design methodology describes how the Smart Dine System is organized and how information flows within it. The system uses a modular design, where each function (like ordering, billing, or menu management) is divided into separate modules that interact through a central database.

The main modules are:

1. Customer Module – Allows users to scan QR codes, view menus, and place orders.
2. Order Module – Handles the transfer of order details to the kitchen and waiter
3. Waiter Module – Displays real-time table orders for serving.
4. Admin Module – Allows adding, updating, and deleting menu items or table data.
5. Database Module – Stores information related to tables, customers, menu items, and transactions.

3.2.1 . INPUT DESIGN

Input design is the process of converting user inputs into a format that the system can process efficiently. In Smart Dine, the input is provided primarily through mobile devices by scanning QR codes and selecting menu items

Key Input Elements:

- Table Name (auto-fetched from QR code)
- Selected Menu Items
- Quantity of Items
- Customer Notes or Special Instructions

Design Objectives:

- Minimize manual entry to avoid errors.
- Use dropdowns, buttons, and digital forms for simple input.
- Validate all inputs before submission.
- Store input data securely in the database.

3.2.2 OUTPUT DESIGN:

Output design involves presenting the processed data in a user-friendly manner. The outputs in Smart Dine are generated for both customers and staff.

System Outputs Include:

- Customer Screen: Order confirmation and bill summary.
- Waiter Dashboard: List of orders with table numbers and order status.
- Kitchen Screen: Display of incoming orders for preparation.
- Admin Reports: Overview of daily orders, table usage, and sales summaries.

Output Design Goals:

- Present accurate and real-time information.
- Maintain clarity and readability.
- Use tables, notifications, and color codes for better visualization.

3.2.3 CODE DESIGN

The coding design defines the programming structure and modular approach for the system. In Smart Dine, each module (Admin, Waiter, Customer) is coded as a separate PHP file for easy maintenance.

Coding Standards Used:

- a. Consistent variable naming and indentation.
- b. Modular structure for reusability.
- c. Separation of front-end (HTML, CSS, JS) and back-end (NODE JS)

3.2.4 DATABASE DESIGN

The database design ensures that all the data related to customers, menu, orders, and tables is stored efficiently with minimal redundancy and maximum data integrity.

Main Tables:

- Table_Info: Stores table name and QR code.
- Menu_Items: Stores details of dishes (item ID, name, price, category).
- Orders: Records all orders placed by customers with table names and timestamps.

- Order_Details: Contains individual item details for each order.
- Admin: Manages admin login details.

Database Design Objectives:

- Reduce redundancy through normalization.
- Ensure data consistency and reliability.
- Provide fast query responses for order management.

A database is a collection of interrelated data stored with minimum redundancy to serve many users quickly and efficiently. The general objective is to make information access easy, quick, inexpensive and flexible for the user. In database design, several specific objectives are considered:

- Controlled redundancy
- Ease of learning and use
- Data independence
- More information at low cost
- Accuracy and integrity
- Recovery from failure
- Privacy and security
- Performance

The scheme is the view that helps us the DBMS decide what data in storage it should act upon as requested by the application program. The subschema is concerned with a relatively small part of scheme. In database design, several views of data must be considered along with the persons who use them. The logical view is what the data look like, regardless of how they stored. The physical view is the way data exists in physical storage. It deals with how data are stored, accessed or related to other data in storage. The logical view are the users view the programmer's view and the overall logical view, called a schema.

3.3 Data Flow Diagram:

The Data Flow Diagram (DFD) illustrates how data moves within the Smart Dine System.

Level 0: Context Diagram

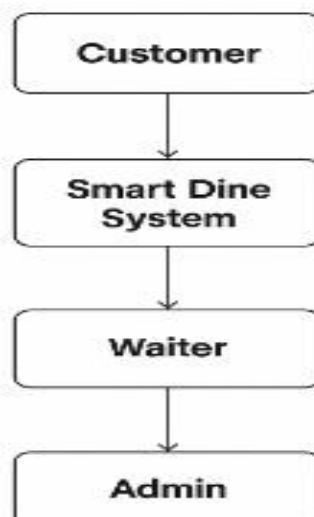
Customer scans QR → Sends request → Smart Dine System → Forwards to Waiter/Kitchen/Admin.

Level 1: Functional Flow

1. Customer scans QR and views the menu.
2. Customer places an order → Order stored in database
3. Waiter and kitchen receive order notifications
4. Admin can monitor and update menu or table information.

3.3.3 Data flow diagram

Data Flow Diagram for Smart Dine



The DFD is an excellent communication tool for analysts to model processes and functional requirement. Used effectively, it is a useful and easy to understand modeling tool. It has broad application and usability across most software development projects. It is easily integrated with data modeling, workflow modeling tools and textual specs. Together with these, it provides analysts and developers with solid models and specs. Alone however, it has limited usability. It is simple and easy to understand by users and can be easily extended and refined with further specification into a physical version for the design and development teams.

The different versions are Context Diagrams (Level 0, Level 1....) Partitioned Diagrams (single process only--one level) functionally decomposed, and level sets of Data Flow Diagrams.

3.3.1 Data Store:

A repository of information. In the physical model, this represents a file, table, etc. In the logical from model, a data store is an object or entity.

3.3.2 Data Flows:

DFDs show the flow of data from external entities into the system, showed hoe the data moved from one process to another, as well as its logical storage. There are only four symbols:

1. Squares representing external entities, which are source or destinations of data.
2. Rounded rectangles representing process, which take data as input, do something to it, and output it.
3. Arrows representing the data flows, which can either, be electronic data or physical items.
4. Open-ended rectangles representing data stores.

There are several common modeling rules for creating DFDs:

All process must have at least one data flow in and one data flow out.

1. All process should modify the incoming data, producing new forms of outgoing data.
2. Each data store must be involved with at least one data flow.
3. Each external entity must be involved with at least one data flow.
4. A data flow must be attached to at least one process.

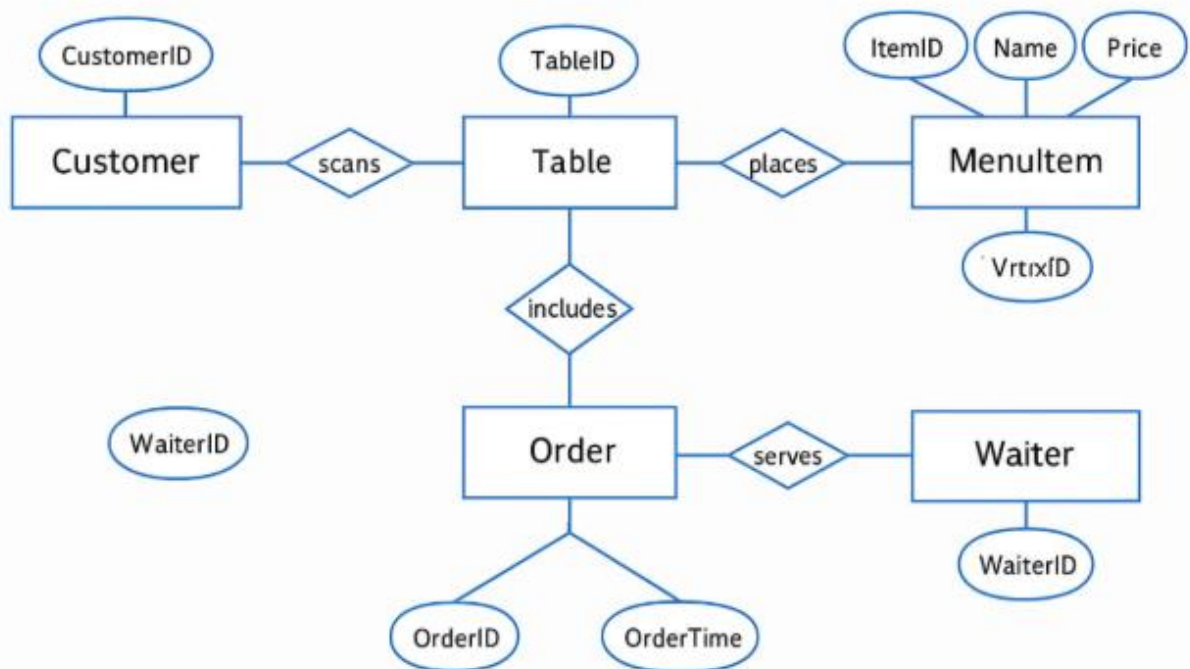
3.3.3 Class Diagram:

The Class Diagram represents the object-oriented structure of the Smart Dine System.
Main Classes:

- Customer

- Table
- Menu Item
- Order
- Waiter
- Admin
- Relationships:
- One Table can have many Orders.
- One Order can include multiple Menu Items.
- Admin manages Menu Items and Tables.

QR Code-Based Restaurant Ordering System

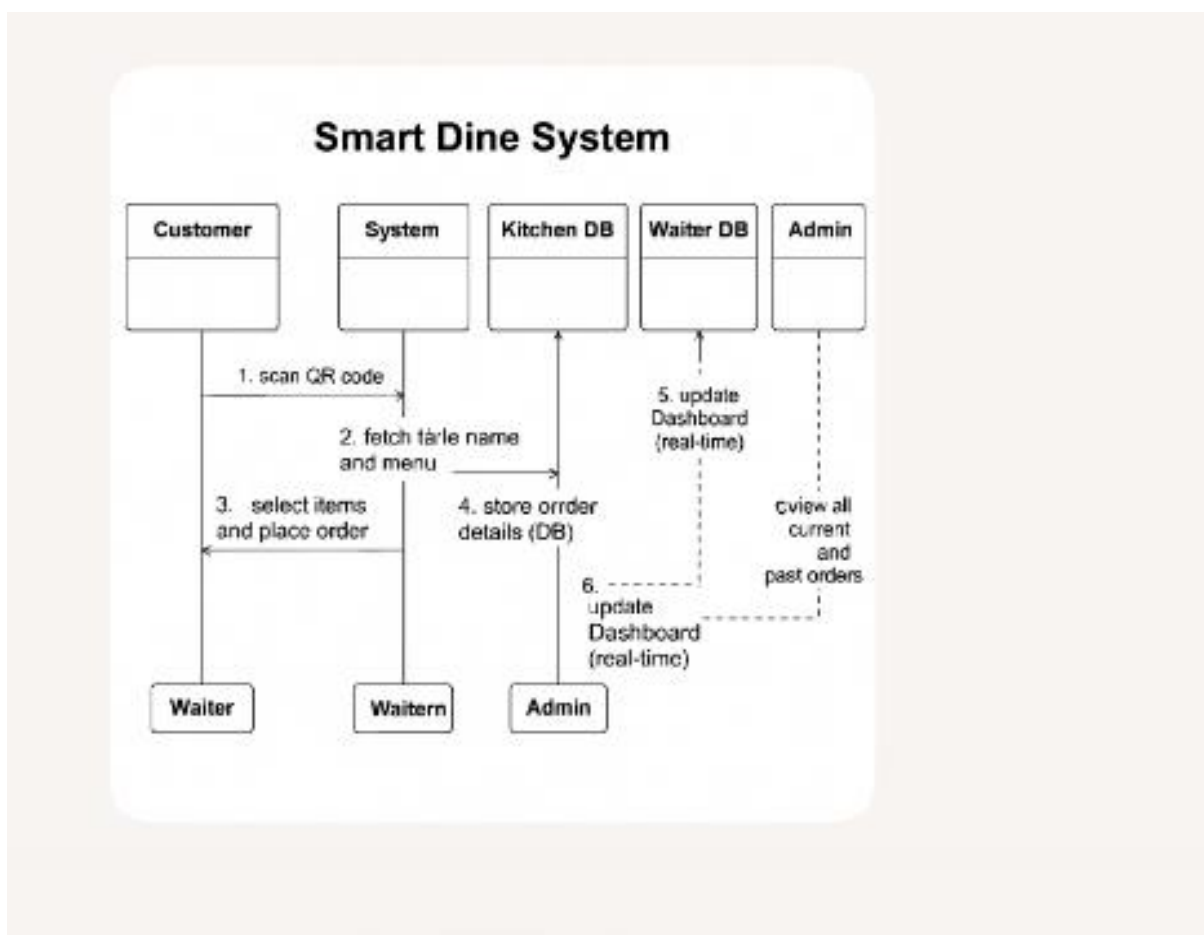


3.3.4 Sequence Diagram:

The Sequence Diagram shows how objects interact in a time sequence.

Steps:

1. Customer scans QR code.
2. System fetches table name and menu.
3. Customer selects items and places order.
4. Order details are stored in the database.
5. Kitchen and waiter dashboards update in real-time.
6. Admin can view all current and past orders.



CHAPTER-4

IMPLEMENTATION

4.1 IMPLEMENTATION:

Index:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>SmartDine - Home</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- Navbar -->
  <header>
    <div class="logo">SmartDine</div>
    <nav id="navbar">
      <a href="index.html" class="active">Home</a>
      <a href="menu.html">Menu</a>
      <a href="premium.html">Premium</a>
      <a href="contact.html">Contact</a>
      <a href="admin.html">Admin</a>
      <a href="cart.html">Cart</a>
    </nav>
    <div class="hamburger" onclick="toggleMenu()">☰</div>
  </header>
  <section class="hero">
    <h2>Welcome to SmartDine</h2>
    <p>Scan • Order • Eat • Pay — Contactless Dining Experience</p>
    <button onclick="window.location.href='menu.html'">View Menu</button>
  </section>
  <!-- Specials Section -->
  <section class="specials">
    <h2>Today's Specials</h2>
    <div class="specials-grid" id="specialsGrid">
      <!-- JS will load items here -->
    </div>
  </section>
  <script src="home.js"></script>
  <!-- Footer -->
  <footer>
    <p>© 2025 SmartDine | Contactless Restaurant Ordering System</p>
  </footer>
</body>
</html>
```



```

sytle sheet
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #fff8f5;
  color: #333;
}
header {
  background: #e63946;
  color: white;
  padding: 15px 30px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  position: sticky;
  top: 0;
  z-index: 1000;
}
header .logo {
  font-size: 24px;
  font-weight: bold;
}
nav {
  display: flex;
  gap: 15px;
}
nav a {
  background: #ffba08;
  color: #333;
  text-decoration: none;
  font-weight: bold;
  padding: 8px 16px;
  border-radius: 6px;
  transition: background 0.3s ease;
}
nav a:hover,
nav a.active {
  background: #faa307;
  color: white;
}
.hamburger {

```

```

display: none;
font-size: 28px;
background: none;
border: none;
color: white;
cursor: pointer;
}

```

Responsive Navbar

```

@media (max-width: 768px) {
  nav {
    display: none;
    flex-direction: column;
    background: #e63946;
    position: absolute;
    top: 60px;
    right: 20px;
    padding: 15px;
    border-radius: 8px;
    box-shadow: 0 4px 8px rgba(0,0,0,0.2);
  }
  nav.show {
    display: flex;
  }
  .hamburger {
    display: block;
  }
}

/* ===== Hero Section (home only) ===== */
.hero {
  background: url('https://images.unsplash.com/photo-1600891964599-f61ba0e24092') no-repeat center center/cover;
  height: 90vh;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  color: white;
  text-align: center;
}
.hero h2 {
  font-size: 48px;

```

```

margin-bottom: 10px;
text-shadow: 2px 2px 6px black;
}
.hero p {
font-size: 20px;
margin-bottom: 20px;
text-shadow: 1px 1px 4px black;
}
.hero button {
background: #ffba08;
border: none;
padding: 15px 30px;
font-size: 18px;
border-radius: 8px;
cursor: pointer;
font-weight: bold;
}
.hero button:hover {
background: #faa307;
}
/* ===== Specials Section (home only) ===== */
.specials {
padding: 50px 20px;
text-align: center;
}
.specials h2 {
font-size: 32px;
margin-bottom: 30px;
color: #e63946;
}
.specials-grid {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
gap: 20px;
padding: 0 20px;
}
.card {
background: white;
border-radius: 10px;
overflow: hidden;
box-shadow: 0 4px 8px rgba(0,0,0,0.1);

```

```

    transition: transform 0.2s;
  }
  .card:hover {
    transform: scale(1.05);
  }
  .card img {
    width: 100%;
    height: 200px;
    object-fit: cover;
  }
  .card h3 {
    margin: 10px;
    color: #333;
  }
  .card p {
    margin: 0 10px 15px;
    color: #777;
  }

```

Menu Page

```

.menu-header {
  text-align: center;
  padding: 40px;
  background: #e63946;
  color: white;
}
.menu-header h1 {
  font-size: 36px;
  margin-bottom: 10px;
}
.search-bar {
  display: flex;
  justify-content: center;
  margin: 20px 0;
}
.search-bar input {
  width: 50%;
  padding: 10px;
  border: 2px solid #ffb300;
  border-radius: 5px;
  font-size: 16px;
}

```

```

.menu {
  padding: 30px;
}
.menu h2 {
  margin-top: 40px;
  color: #e63946;
  border-bottom: 2px solid #e63946;
  display: inline-block;
  padding-bottom: 5px;
}
.menu-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  gap: 20px;
  margin-top: 20px;
}
.menu-item {
  background: white;
  padding: 15px;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
  transition: transform 0.2s;
}
.menu-item:hover {
  transform: scale(1.03);
}
.menu-item h3 {
  display: flex;
  justify-content: space-between;
  margin-bottom: 10px;
  font-size: 18px;
}
.menu-item p {
  color: #555;
  margin-bottom: 5px;
}
.menu-item small {
  color: #888;
}
.menu-item textarea {
  width: 100%;

```

```

height: 40px;
margin-top: 10px;
padding: 5px;
border: 1px solid #ccc;
border-radius: 5px;
resize: none;
}
.menu-item button {
background: #ffba08;
color: #333;
border: none;
padding: 10px;
margin-top: 10px;
border-radius: 5px;
cursor: pointer;
font-weight: bold;
width: 100%;
}
.menu-item button:hover {
background: #faa307;
color: white;
}
footer {
background: #333;
color: white;
text-align: center;
padding: 20px;
margin-top: 40px;
}
.categories {
padding: 50px 20px;
text-align: center;
}
.categories h2 {
font-size: 32px;
margin-bottom: 30px;
color: #e63946;
}
.category-grid {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));

```

```

    gap: 25px;
    padding: 0 20px;
}
.category-card {
    background: white;
    border-radius: 12px;
    overflow: hidden;
    box-shadow: 0 4px 8px rgba(0,0,0,0.1);
    cursor: pointer;
    transition: transform 0.2s ease, box-shadow 0.2s ease;
}
.category-card:hover {
    transform: scale(1.05);
    box-shadow: 0 6px 12px rgba(0,0,0,0.15);
}
.category-card img {
    width: 100%;
    height: 180px;
    object-fit: cover;
}
.category-card h3 {
    margin: 15px;
    font-size: 20px;
    color: #333;
}
.nav-links {
    list-style: none;
    display: flex;
    gap: 20px;
}
.hamburger {
    display: none;
}
@media (max-width: 768px) {
    .nav-links {
        display: none;
        flex-direction: column;
        background: #e63946;
        position: absolute;
        top: 60px;
        right: 20px;
    }
}

```

```

width: 200px;
padding: 15px;
border-radius: 8px;
}
.nav-links.active {
display: flex;
}
.hamburger {
display: block;
}
}
script.js
function showAlert() {
alert("Menu page coming soon! ");
}
function goToMenu() {
window.location.href = "menu.html";
}
const hamburger = document.getElementById("hamburger");
const navLinks = document.getElementById("navLinks");

hamburger.addEventListener("click", () => {
navLinks.classList.toggle("active");
});
menu.html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>SmartDine Menu</title>
<link rel="stylesheet" href="menu.css">
</head>
<body>
<!-- Navbar -->
<header>
<h1>SmartDine</h1>
<nav>
<a href="index.html">Home</a>
<a href="menu.html" class="active">Menu</a>
<a href="premium.html">Premium</a>

```



```

    <a href="contact.html">Contact</a>
    <a href="admin.html">Admin</a>
    <a href="cart.html">Cart </a>
  </nav>
</header>
<!-- Category Section -->
<section class="categories">
  <h2>Explore Categories</h2>
  <div class="category-grid">
    <div class="category-card" onclick="showCategory('soup')">
      
      <h3>Soup</h3>
    </div>
    <div class="category-card" onclick="showCategory('tandoori')">
      
      <h3>Tandoori & Kebab</h3>
    </div>
    <div class="category-card" onclick="showCategory('barbecue')">
      
      <h3>Gills & Barbecue</h3>
    </div>
    <div class="category-card" onclick="showCategory('starters')">
      
      <h3>Starters</h3>
    </div>
    <div class="category-card" onclick="showCategory('vegstarters')">
      
      <h3>Veg Starters</h3>
    </div>
    <div class="category-card" onclick="showCategory('seafood')">
      
      <h3>Seafood</h3>
    </div>
    <div class="category-card" onclick="showCategory('biryani')">
      

```

```

    <h3>Biryani</h3>
  </div>
  <div class="category-card" onclick="showCategory('traditional')">
    
    <h3>Traditional Dishes</h3>
  </div>
  <div class="category-card" onclick="showCategory('graviesveg')">
    
    <h3>Gravies (Veg)</h3>
  </div>
  <div class="category-card" onclick="showCategory('graviesnonveg')">
    
    <h3>Gravies (Non-Veg)</h3>
  </div>
  <div class="category-card" onclick="showCategory('rice')">
    
    <h3>Rice & Noodles</h3>
  </div>
  <div class="category-card" onclick="showCategory('sizzler')">
    
    <h3>Sizzler</h3>
  </div>
  <div class="category-card" onclick="showCategory('breads')">
    
    <h3>Indian Breads</h3>
  </div>
  <div class="category-card" onclick="showCategory('eggs')">
    
    <h3>Eggs</h3>
  </div>
  <div class="category-card" onclick="showCategory('quick')">
    
    <h3>Quick Bites</h3>
  </div>

```

```

</div>
<div class="category-card" onclick="showCategory('desserts')">
  
  <h3>Desserts</h3>
</div>
<div class="category-card" onclick="showCategory('falooda')">
  
  <h3>Falooda</h3>
</div>
<div class="category-card" onclick="showCategory('juice')">
  
  <h3>Fresh Juice</h3>
</div>
<div class="category-card" onclick="showCategory('lassi')">
  
  <h3>Lassi</h3>
</div>
<div class="category-card" onclick="showCategory('shakes')">
  
  <h3>Shakes</h3>
</div>
<div class="category-card" onclick="showCategory('mojito')">
  
  <h3>Mojito</h3>
</div>
<div class="category-card" onclick="showCategory('hotdrinks')">
  
  <h3>Hot Drinks & Lemonade</h3>
</div>
<div class="category-card" onclick="showCategory('icecream')">
  
  <h3>Ice Cream</h3> </div>
<div class="category-card" onclick="showCategory('cakes')">
  

<h3>Cakes</h3> </div> </div></section>

<section id="dishList" style="display:none; padding:30px;">

<h2 id="dishTitle"></h2>

<div class="menu-container" id="dishContainer"></div>

<button class="back-btn" onclick="backToCategories()">← Back</button>

</section>

<script>

```
const dishes = { soup: [{ name: "Chicken Clear Soup", price: 90,
img:"https://www.whiskaffair.com/wp-content/uploads/2016/08/Clear-Chicken-Soup-2-2.jpg" },
 { name: "Manchow Soup", price: 90, img:
"https://tse3.mm.bing.net/th/id/OIP.N6w5JPAS4iu2vreE0Gw9oAHaJb?pid=Api&P=0&h=180" },
 { name: "Sweet Corn Soup", price: 90, img:
"https://tse2.mm.bing.net/th/id/OIP.bwRPqUfAsRn7VeX9pk4y8wHaJQ?pid=Api&P=0&h=180" },
 { name: "Hot & Sour Soup", price: 100, img:
"https://static.toiimg.com/thumb/53109843.cms?imgsize=165137&width=800&height=800" }
], tandoori: [{ name: "Tandoori Chicken Full", price: 440, img:
"https://tse2.mm.bing.net/th/id/OIP.1j118t-w5dreMw0ydoXNagHaE8?pid=Api&P=0&h=180" },
 { name: "Tandoori Chicken Half", price: 220, img: "https://tse2.mm.bing.net/th/id/OIP.1j118t-
w5dreMw0ydoXNagHaE8?pid=Api&P=0&h=180" },
 { name: "Chicken Tikka (7pcs)", price: 180, img:
"https://tse4.mm.bing.net/th/id/OIP.vMIBYE5d1PIJVOJuRgaV3AHaGt?pid=Api&P=0&h=180" },
 { name: "Paneer Tikka", price: 150, img:
"https://tse2.mm.bing.net/th/id/OIP.DH0KMPXHqWYW28hEc7OXLQHaE8?pid=Api&P=0&h=180"
}
], barbecue: [{ name: "Grill Chicken Full", price: 440, img:
"https://tse3.mm.bing.net/th/id/OIP.6NkgjGMOV_OPA_HSMVsFMwHaHa?pid=Api&P=0&h=180" },
 { name: "Alfaham BBQ Full", price: 440, img:
"https://i.ytimg.com/vi/HVi7xxQZDRQ/maxresdefault.jpg" },
 { name: "Alfaham BBQ Half", price: 220, img:
"https://i.ytimg.com/vi/HVi7xxQZDRQ/maxresdefault.jpg" }
], starters: [{ name: "Chicken Kebab (7pcs)", price: 180, img:
"https://tse3.mm.bing.net/th/id/OIP.qzLFYJcJhtW95STUI6tltAHaHa?pid=Api&P=0&h=180" },
 { name: "Chicken 65 (8pcs)", price: 100, img:
"https://tse3.mm.bing.net/th/id/OIP.YPHaS9Emmf2UslGqRZ3P_wHaE8?pid=Api&P=0&h=180" },
 { name: "Pepper Chicken", price: 150, img:
"https://tse3.mm.bing.net/th/id/OIP.Pm3mJvp4arYILt3nYUvd9AHaLH?pid=Api&P=0&h=180" }
], vegstarters: [{ name: "Chilli Honey Potato", price: 130, img:
"https://tse3.mm.bing.net/th/id/OIP.Fia0nJIy-Bc2PwffRQmcwgHaHa?pid=Api&P=0&h=180" },
 { name: "Veg Manchurian", price: 140, img:"https://tse3.mm.bing.net/th/id/OIP.Fia0nJIy-
Bc2PwffRQmcwgHaHa?pid=Api&P=0&h=180" },
 { name: "Paneer Chilli", price: 160, img: "https://www.cookwithmanali.com/wp-
content/uploads/2016/01/Chilli-Paneer-Restaurant-Style.jpg" }
],seafood: [{ name: "Fish Fry", price: 180, img: "https://tse2.mm.bing.net/th/id/OIP.tQKg49I-
6gwXqe3ttDNzZgHaEo?pid=Api&P=0&h=180" }, { name: "Fish Curry", price: 190, img:
```

"https://thumbs.dreamstime.com/b/traditional-kerala-fish-curry-coconut-milk-fresh-mangoes-82241970.jpg" }, { name: "Prawn Curry", price: 200, img: "https://www.indianhealthyrecipes.com/wp-content/uploads/2023/02/paneer-curry-dhaba-style.webp" }

], biryani: [ { name: "Chicken Biryani", price: 220, img: "https://tse1.mm.bing.net/th/id/OIP.UI6wfhsE9XEBC23wyzXw7QHaHa?pid=Api&P=0&h=180" }, { name: "Mutton Biryani", price: 290, img: "https://tse4.mm.bing.net/th/id/OIP.qTzBlm8Nyc4KQofktP11AHaE8?pid=Api&P=0&h=180" }, { name: "Veg Biryani", price: 200, img: "https://tse2.mm.bing.net/th/id/OIP.LadujoU81UAUhQjy9gEIUwHaHa?pid=Api&P=0&h=180" } ], traditional: [ { name: "Mutton Sukka", price: 200, img: "https://www.sharmispassions.com/mutton-chukka-recipe-mutton-dry-fry/" }, { name: "Kaadae Phall", price: 170, img: "https://culinaryambition.com/thai-beef-with-holy-basil/" }, { name: "Chicken Curry", price: 180, img: "https://www.allrecipes.com/recipe/46822/indian-chicken-curry-ii/" } ], graviesveg: [ { name: "Paneer Butter Masala", price: 170, img: "https://www.cookwithmanali.com/paneer-butter-masala-recipe/" }, { name: "Paneer Kadai", price: 170, img: "https://www.cookwithmanali.com/kadai-paneer/" }, { name: "Veg Kolhapuri", price: 160, img: "https://www.cookingcarnival.com/veg-kolhapuri/" } ], graviesnonveg: [ { name: "Butter Chicken", price: 180, img: "https://www.cookwithmanali.com/wp-content/uploads/2020/08/Butter-Chicken-500x500.jpg" }, { name: "Chicken Masala", price: 180, img: "https://www.indianhealthyrecipes.com/wp-content/uploads/2021/08/chicken-masala.jpg" }, { name: "Mutton Masala", price: 250, img: "https://www.indianhealthyrecipes.com/wp-content/uploads/2021/07/mutton-masala.jpg" } ], rice: [ { name: "Veg Fried Rice", price: 100, img: "https://www.indianhealthyrecipes.com/wp-content/uploads/2021/07/veg-fried-rice-recipe.jpg" }, { name: "Chicken Fried Rice", price: 110, img: "https://www.indianhealthyrecipes.com/wp-content/uploads/2021/07/chicken-fried-rice.jpg" }, { name: "Prawn Fried Rice", price: 160, img: "https://www.kannammacooks.com/wp-content/uploads/prawn-fried-rice-recipe.jpg" } ], sizzler: [ { name: "Chicken Sizzler", price: 250, img: "https://upload.wikimedia.org/wikipedia/commons/8/84/Chicken\_sizzler.jpg" }, { name: "Paneer Sizzler", price: 220, img: "https://upload.wikimedia.org/wikipedia/commons/4/4e/Paneer\_sizzler.jpg" } ], breads: [ { name: "Romali Roti", price: 35, img: "https://upload.wikimedia.org/wikipedia/commons/2/2e/Rumali\_roti.jpg" }, { name: "Parotta", price: 25, img: "https://upload.wikimedia.org/wikipedia/commons/3/35/Parotta.jpg" }, { name: "Butter Naan", price: 40, img: "https://upload.wikimedia.org/wikipedia/commons/1/1f/Butter\_Naan.jpg" } ], eggs: [ { name: "Egg Masala", price: 110, img: "https://upload.wikimedia.org/wikipedia/commons/f/fb/Egg\_curry.jpg" }, { name: "Egg Curry", price: 120, img: "https://upload.wikimedia.org/wikipedia/commons/3/3f/Egg\_curry\_with\_rice.jpg" },

```

 { name: "Omelette", price: 15, img:
"https://upload.wikimedia.org/wikipedia/commons/7/71/Omelette_with_vegetables.jpg" }
], quick: [{ name: "Chicken Shawarma", price: 70, img:
"https://upload.wikimedia.org/wikipedia/commons/b/b0/Chicken_Shawarma.jpg" },
 { name: "Spring Roll", price: 100, img:
"https://upload.wikimedia.org/wikipedia/commons/7/76/Spring_rolls.jpg" },
 { name: "French Fries", price: 80, img:
"https://upload.wikimedia.org/wikipedia/commons/4/41/Fries_2.jpg" }
], desserts: [{ name: "Rasmalai (5pcs)", price: 120, img:
"https://upload.wikimedia.org/wikipedia/commons/d/d2/Rasmalai_-_Bengali_Sweet.jpg" },
 { name: "Gulab Jamun with Ice Cream", price: 120, img:
"https://upload.wikimedia.org/wikipedia/commons/b/b8/Gulab_Jamun.jpg" },
 { name: "Rasgulla (5pcs)", price: 100, img:
"https://upload.wikimedia.org/wikipedia/commons/0/00/Rasgulla.jpg" }
], falooda: [{ name: "Royal Falooda", price: 100, img:
"https://upload.wikimedia.org/wikipedia/commons/0/09/Falooda.jpg" },
 { name: "Ahmedia Spl Falooda", price: 130, img:
"https://upload.wikimedia.org/wikipedia/commons/a/a7/Falooda_with_ice_cream.jpg" }
], juice: [{ name: "Apple Juice", price: 80, img:
"https://upload.wikimedia.org/wikipedia/commons/f/f6/Apple_juice_01.jpg" },
 { name: "Pomegranate Juice", price: 80, img:
"https://upload.wikimedia.org/wikipedia/commons/9/9a/Pomegranate_juice_2.jpg" },
 { name: "Watermelon Juice", price: 70, img:
"https://upload.wikimedia.org/wikipedia/commons/5/5a/Watermelon_juice_glass.jpg" }
], lassi: [{ name: "Sweet Lassi", price: 50, img:
"https://upload.wikimedia.org/wikipedia/commons/1/1a/Sweet_Lassi.jpg" },
 { name: "Mango Lassi", price: 60, img:
"https://upload.wikimedia.org/wikipedia/commons/d/d7/Mango_Lassi.jpg" }
], shakes: [{ name: "Oreo Shake", price: 100, img:
"https://upload.wikimedia.org/wikipedia/commons/9/9f/Cookies_and_cream_milkshake.jpg" },
 { name: "Dry Fruits Shake", price: 120, img:
"https://upload.wikimedia.org/wikipedia/commons/a/a4/Milkshake_glass.jpg" },
 { name: "Chocolate Shake", price: 90, img:
"https://upload.wikimedia.org/wikipedia/commons/f/f7/Chocolate_milkshake.jpg" }
], mojito: [{ name: "Mint Mojo", price: 80, img:
"https://upload.wikimedia.org/wikipedia/commons/7/7b/Mojito_in_a_glass.jpg" }, { name: "Strawberry
Mojo", price: 80, img: "https://upload.wikimedia.org/wikipedia/commons/1/17/Strawberry_Mojito.jpg"
}
], hotdrinks: [{ name: "Lime Juice (Sweet & Salt)", price: 50, img:
"https://upload.wikimedia.org/wikipedia/commons/d/d1/Lemonade_glass.jpg" },
 { name: "Tea", price: 20, img:
"https://upload.wikimedia.org/wikipedia/commons/7/7e/Indian_tea_glass.jpg" }, { name: "Coffee",
price: 60, img: "https://upload.wikimedia.org/wikipedia/commons/4/45/A_cup_of_coffee.JPG" }
], icecream: [{ name: "Chocolate Sundae", price: 120, img:
"https://upload.wikimedia.org/wikipedia/commons/f/f8/Chocolate_sundae.jpg" },
 { name: "Fruit Salad with Ice Cream", price: 120, img:

```

```

"https://upload.wikimedia.org/wikipedia/commons/2/23/Fruit_salad_with_ice_cream.jpg" },
 { name: "Vanilla Scoop", price: 60, img:
"https://upload.wikimedia.org/wikipedia/commons/5/56/Vanilla_ice_cream_cone.jpg" }
],cakes: [{ name: "Death by Chocolate", price: 150, img:
"https://upload.wikimedia.org/wikipedia/commons/8/84/Chocolate_cake_slice.jpg" },
 { name: "Hot Chocolate Cake", price: 100, img:
"https://upload.wikimedia.org/wikipedia/commons/f/fb/Chocolate_cake_with_frosting.jpg" }]
};

function showCategory(category) {
 document.querySelector(".categories").style.display = "none";
 document.getElementById("dishList").style.display = "block";
 document.getElementById("dishTitle").textContent =
 category.charAt(0).toUpperCase() + category.slice(1);
 let container = document.getElementById("dishContainer");
 container.innerHTML = "";
 dishes[category].forEach((dish, index) => {
 container.innerHTML += `
 <div class="menu-card">
 <div class="menu-info"> <h3>${dish.name}</h3>
 <p>₹${dish.price}</p>
 <div class="quantity">
 <button onclick="decreaseQty('${category}', ${index})">-</button>
 1
 <button onclick="increaseQty('${category}', ${index})">+</button>
 </div>
 <button class="add-btn" onclick="addToCart('${dish.name}', ${dish.price}, '${category}',
 ${index})">Add to Cart</button></div>
 </div> `;
 });}

function backToCategories() {
 document.querySelector(".categories").style.display = "block";
 document.getElementById("dishList").style.display = "none";
}

function decreaseQty(category, index) {
 let qtyEl = document.getElementById(`qty-${category}-${index}`);
 let qty = parseInt(qtyEl.textContent);
 if (qty > 1) qtyEl.textContent = qty - 1; }

function increaseQty(category, index) {
 let qtyEl = document.getElementById(`qty-${category}-${index}`);
 let qty = parseInt(qtyEl.textContent);
 qtyEl.textContent = qty + 1;
}

```

```

function addToCart(name, price, category, index) {
 let qty = parseInt(document.getElementById(`qty-${category}-${index}`).textContent);
 let cart = JSON.parse(localStorage.getItem("cart")) || [];
 cart.push({ name, price, qty });
 localStorage.setItem("cart", JSON.stringify(cart));
 alert(qty + " × " + name + " added to cart ");
}

function addToCart(name, price, category, index) {
 let qty = parseInt(document.getElementById(`qty-${category}-${index}`).textContent);
 let cart = JSON.parse(localStorage.getItem("cart")) || []
 const existingItemIndex = cart.findIndex(item => item.name === name)
 if (existingItemIndex !== -1) {
 cart[existingItemIndex].qty += qty;
 } else {
 cart.push({ name, price, qty, note: "", category: category, menuItemId: `${category}-${index}`
 });
 }
 localStorage.setItem("cart", JSON.stringify(cart));
 alert(qty + " × " + name + " added to cart ")
 updateCartCount();
}

function updateCartCount() {
 const cart = JSON.parse(localStorage.getItem("cart")) || [];
 const totalItems = cart.reduce((sum, item) => sum + item.qty, 0)
 const cartLink = document.querySelector('a[href="cart.html"]');
 if (cartLink) {
 cartLink.textContent = `Cart (${totalItems})`; } }
 updateCartCount();</script>
</body>
</html>
menu.js
document.getElementById("search").addEventListener("keyup", function() {
 let filter = this.value.toLowerCase();
 let items = document.querySelectorAll(".menu-item");
 items.forEach(item => {
 let text = item.textContent.toLowerCase();
 item.style.display = text.includes(filter) ? "block" : "none";
 });});
premium.html
<!DOCTYPE html>
<html lang="en">
<head>

```



```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Premium Membership - SmartDine</title>
<link rel="stylesheet" href="premium.css">
</head>
<body>
 <!-- Navbar -->
 <nav class="navbar">
 <div class="logo">SmartDine Premium</div>

 Home
 Menu
 Premium

 </nav>
 <header class="hero">
 <h1> Become a Premium Member</h1>
 <p>Exclusive perks, priority service & rewards for our loyal guests</p>
 <div class="hero-buttons">
 <button onclick="showLogin()">Login</button>
 <button onclick="showRegister()">Sign Up</button>
 </div>
 </header>
 <section id="auth-section" class="card">
 <h2 id="form-title"> Login</h2>
 <form id="auth-form">
 <input type="text" id="name" placeholder="Name (only for Sign Up)" style="display:none;">
 <input type="email" id="email" placeholder="Email" required>
 <input type="password" id="password" placeholder="Password" required>
 <button type="submit">Continue</button>
 </form>
 <p id="toggle-form">Don't have an account? Sign
up</p>
 </section>
 <section id="dashboard" class="hidden">
 <h2> Welcome back, </h2>
 <!-- Perks -->
 <div class="perks">
 <div class="perk-card"> <h3>10% Discounts</h3><p>On all orders</p></div>
 <div class="perk-card"> <h3>Priority Seating</h3><p>Skip the wait</p></div>
 <div class="perk-card"> <h3>Exclusive Menu</h3><p>Hidden premium dishes</p></div>
 </div>
 </section>

```

```

</div>
<div class="reservation card">
 <h3> Reserve a Table</h3>
 <form id="reservation-form">
 <input type="date" id="res-date" required>
 <input type="time" id="res-time" required>
 <input type="number" id="res-guests" placeholder="Guests" required>
 <textarea id="res-notes" placeholder="Special requests"></textarea>
 <button type="submit">Reserve</button>
 </form>
 <h4>Your Reservations</h4>
 <ul id="reservation-list">
</div>
<!-- Profile -->
<div class="profile card">
 <h3> My Profile</h3>
 <p>Email: </p>
 <p>Loyalty ID: </p>
 <button onclick="logout()">Logout</button>
</div>
</section>
<script src="premium.js"></script>
</body>
</html>

```

```

premium.js
let users = JSON.parse(localStorage.getItem("premiumUsers")) || [];
let reservations = JSON.parse(localStorage.getItem("reservations")) || [];
let currentUser = null;
function showLogin() {
 document.getElementById("form-title").innerText = " Login";
 document.getElementById("name").style.display = "none";
}
function showRegister() {
 document.getElementById("form-title").innerText = " Sign Up";
 document.getElementById("name").style.display = "block";
}
document.getElementById("auth-form").addEventListener("submit", function(e) {
 e.preventDefault();
 const name = document.getElementById("name").value;
 const email = document.getElementById("email").value;
 const password = document.getElementById("password").value;

```

```

if (document.getElementById("form-title").innerText.includes("Sign Up")) {
 users.push({ name, email, password });
 localStorage.setItem("premiumUsers", JSON.stringify(users));
 alert("Registered! Please login now.");
 showLogin();
} else {
 let user = users.find(u => u.email === email && u.password === password);
 if (user) {
 currentUser = user;
 localStorage.setItem("currentUser", JSON.stringify(user));
 loadDashboard();
 } else {
 alert("Invalid credentials!");
 }
});
function loadDashboard() {
 document.getElementById("auth-section").classList.add("hidden");
 document.getElementById("dashboard").classList.remove("hidden");
 currentUser = JSON.parse(localStorage.getItem("currentUser"));
 document.getElementById("memberName").innerText = currentUser.name;
 document.getElementById("profileEmail").innerText = currentUser.email;
 document.getElementById("loyaltyId").innerText = "SD" +
currentUser.email.slice(0,3).toUpperCase() + "123";
 renderReservations();
}
function logout() {
 localStorage.removeItem("currentUser");
 currentUser = null;
 document.getElementById("dashboard").classList.add("hidden");
 document.getElementById("auth-section").classList.remove("hidden");
}
document.getElementById("reservation-form").addEventListener("submit", function(e) {
 e.preventDefault();
 const res = {
 date: document.getElementById("res-date").value, time: document.getElementById("res-
time").value, guests: document.getElementById("res-guests").value,
 notes: document.getElementById("res-notes").value, user: currentUser.email
 };
 reservations.push(res);
 Storage.setItem("reservations", JSON.stringify(reservations));
 renderReservations();
});

```

```

function renderReservations() {
 const list = document.getElementById("reservation-list");
 list.innerHTML = "";
 reservations.filter(r => r.user === currentUser.email).forEach(r => {
 const li = document.createElement("li");
 li.textContent = `${r.date} ${r.time} - ${r.guests} guests (${r.notes})`;
 list.appendChild(li);
 });
}
if (Storage.getItem("currentUser")) {
 loadDashboard();
}

```

contact.html

```

<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Contact - SmartDine</title>
 <link rel="stylesheet" href="contact.css">
</head>
<body>
 <!-- Navbar -->
 <header>
 <h1>SmartDine</h1>
 <nav>
 Home
 Menu
 Premium
 Contact
 Cart
 </nav>
 </header>
 <!-- Hero Section -->
 <section class="contact-hero">
 <h2>Get in Touch</h2>
 <p>We're here to help — whether it's a quick query, reservation issue, or just to say hi!</p>
 </section>
 <!-- Contact Info Section -->
 <section class="contact-info">
 <div class="info-card">
 <h3>Hours of Operation</h3>

```

```

 <p>Mon – Sun: 10:00 AM – 11:00 PM</p>
 <p>Excluding Public Holidays</p>
</div>
<div class="info-card">
 <h3> Call Us</h3>
 <p>+91 97899 57524</p>
 <p>+91 73585 70267</p>
</div>
<div class="info-card">
 <h3> Email Us</h3>
 <p>nazeefasuman@gmail.com</p>
 <p>mubash322@gmail.com</p>
</div>
<div class="info-card">
 <h3> Table Reservation</h3>
 <p>Reserve your table in advance for a hassle-free dining experience</p>
</div>
</section>
<!-- Map Section -->
<section class="map-section">
 <h2> Find Us</h2>
 <iframe
src="https://www.google.com/maps/embed?pb=!1m18!1m12!1m3!1d3820.43830624782!2d78.623876
31500036!3d12.678612789858566!2m3!1f0!2f0!3f0!3m2!1i1024!2i768!4f13.1!3m3!1m2!1s0x3babfc3
ce4b0f8d5%3A0x1a1cdd1c75f74b6e!2sIslamiah%20Women's%20Arts%20%26%20Science%2
0College!5e0!3m2!1sen!2sin!4v1700000000000"
width="100%"
height="300"
style="border:0;"
allowfullscreen=""
loading="lazy">
</iframe>
</section>
<!-- Footer -->
<footer>
 <p>© 2025 SmartDine | Contactless Restaurant Ordering System</p>
</footer>
</body>
</html>
admin.html
<!DOCTYPE html>
<html lang="en">

```

```

<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Admin Panel - SmartDine</title>
 <link rel="stylesheet" href="admin.css">
</head>
<body>
 <header>
 <h1>SmartDine Admin</h1>
 <nav>
 Home
 Menu
 Premium
 Contact
 Admin
 Logout
 </nav>
 </header>
 <section class="admin-dashboard">
 <h2>Manage Today's Specials</h2>
 <form id="specialsForm">
 <label>Dish Name:</label>
 <input type="text" id="dishName" required>
 <label>Price:</label>
 <input type="text" id="dishPrice" required>
 <label>Description:</label>
 <input type="text" id="dishDesc" required>
 <label>Image URL:</label>
 <input type="text" id="dishImg" required>
 <button type="submit">Add Special</button>
 </form>
 <h3>Current Specials</h3>
 <ul id="specialsList">
 </section>
 <script src="admin.js"></script>
</body>
</html>
admin.js
if (localStorage.getItem("isAdminLoggedIn") !== "true") {
 window.location.href = "admin-login.html";
}

```

```

const form = document.getElementById("specialsForm");
const specialsList = document.getElementById("specialsList");
let specials = JSON.parse(localStorage.getItem("specials")) || [];
renderSpecials();
form.addEventListener("submit", function(e) {
 e.preventDefault();
 const dish = {
 name: document.getElementById("dishName").value,
 price: document.getElementById("dishPrice").value,
 desc: document.getElementById("dishDesc").value,
 img: document.getElementById("dishImg").value
 };
 specials.push(dish);
 Storage.setItem("specials", JSON.stringify(specials));form.reset(); renderSpecials();
});
function renderSpecials() {
 specialsList.innerHTML = "";
 specials.forEach((dish, index) => {
 let li = document.createElement("li");
 li.innerHTML = `

 \${dish.name} - \${dish.price}

 <small>\${dish.desc}</small>

 <button onclick="deleteSpecial(\${index})">Delete</button>`;
 specialsList.appendChild(li); });
}
function deleteSpecial(index) {
 specials.splice(index, 1);
 localStorage.setItem("specials", JSON.stringify(specials));
 renderSpecials();
}
function logout() {
 localStorage.removeItem("isAdminLoggedIn");
 window.location.href = "admin-login.html";
}

```

Cart.js

```

function loadCart() {
 let cart = JSON.parse(localStorage.getItem("cart")) || [];
 let container = document.getElementById("cartContainer");
 let total = 0;
 container.innerHTML = "";

```

```

if (cart.length === 0) {
 container.innerHTML = "<p>Your cart is empty!</p>";
 document.getElementById("cartTotal").textContent = "Total: ₹0";
 return;
}
cart.forEach((item, index) => {
 total += item.price * item.qty;
 container.innerHTML += `
 <div class="cart-item">
 <h3>${item.name}</h3>
 <p>₹${item.price * item.qty}</p>
 <div class="quantity">
 <button onclick="decreaseQty(${index})">-</button>
 ${item.qty}
 <button onclick="increaseQty(${index})">+</button>
 </div>
 <textarea
 placeholder="Special instructions (extra cheese, no ghee...)"
 oninput="updateNote(${index}, this.value)"
 >${item.note || ""}</textarea>
 <button class="remove-btn" onclick="removeItem(${index})"> Remove</button>
 </div>
 `;
});
document.getElementById("cartTotal").textContent = "Total: ₹" + total;
}

function decreaseQty(index) {
 let cart = JSON.parse(Storage.getItem("cart")) || [];
 if (cart[index].qty > 1) cart[index].qty--;
 localStorage.setItem("cart", JSON.stringify(cart));
 loadCart();
}

function increaseQty(index) {
 let cart = JSON.parse(Storage.getItem("cart")) || [];
 cart[index].qty++;
 localStorage.setItem("cart", JSON.stringify(cart));
 loadCart();
}

function updateNote(index, note) {
 let cart = JSON.parse(Storage.getItem("cart")) || [];
 cart[index].note = note;
 localStorage.setItem("cart", JSON.stringify(cart));
}

```



```

}
function removeItem(index) {
 let cart = JSON.parse(localStorage.getItem("cart")) || [];
 cart.splice(index, 1);
 localStorage.setItem("cart", JSON.stringify(cart));
 loadCart();
}
async function placeOrder() {
 let cart = JSON.parse(localStorage.getItem("cart")) || [];
 if (cart.length === 0) {
 alert("Your cart is empty!");
 return;
 }
 const customerName = prompt("Enter your name:");
 const customerPhone = prompt("Enter your phone number:");
 const customerEmail = prompt("Enter your email (optional):");
 const tableNumber = prompt("Enter table number (optional):");
 if (!customerName || !customerPhone) {
 alert("Name and phone number are required!");
 return;
 }
 const orderData = {
 items: cart.map(item => ({
 menuItem: item.menuItemId || 'temp-id', name: item.name, price: item.price, quantity: item.qty,
 specialInstructions: item.note || "
 tableNumber || "
 }, notes: "
 });
 const modal = document.getElementById('paymentModal');
 const modalTotal = document.getElementById('modalTotal');
 const totalAmount = cart.reduce((s, i) => s + i.price * i.qty, 0);
 modalTotal.textContent = `Total Amount: ₹${totalAmount}`;
 modal.classList.remove('hidden');
 document.querySelectorAll('.pay-method').forEach(btn => {
 btn.onclick = async function() {
 const method = this.dataset.method;
 modal.classList.add('hidden');
 await submitOrderWithMethod(method);
 };
 });
}

```

```

document.getElementById('cancelPayment').onclick = function() {
 modal.classList.add('hidden');
};
}
async function submitOrderWithMethod(method) {
 const pending = window.__pendingOrder;
 if (!pending) return;
 const orderData = {
 items: pending.cart.map(item => ({
 menuItem: item.menuItemId || 'temp-id',
 name: item.name, price: item.price, quantity: item.qty, specialInstructions: item.note || "
 })),
 customerInfo: pending.customerInfo, paymentMethod: method, notes: pending.notes
 };
 try {
 const response = await fetch('http://localhost:3000/api/orders', {
 method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(orderData)
 });
 const data = await response.json();
 if (response.ok) {
 if (method === 'upi') {
 alert('Redirecting to UPI/Google Pay (demo)');
 }
 alert(`✅ Order placed successfully!\nOrder Number: ${data.order.orderNumber}\nTotal:
₹${data.order.totalAmount}\nEstimated Time: ${data.order.estimatedTime} minutes`);
 Storage.removeItem('cart');
 window.pendingOrder = null;
 window.location.href = 'menu.html';
 } else {
 alert(`❌ Order failed: ${data.message}`);
 }
 } catch (err) {
 console.error('Order error:', err);
 alert(`❌ Cannot connect to server. Please try again later.`);
 }
}
loadCart();
payment.js
const onlinePaymentBtn = document.getElementById('onlinePayment');
const cashPaymentBtn = document.getElementById('cashPayment');
const onlineOptions = document.getElementById('onlineOptions');

```

```

const cashMessage = document.getElementById('cashMessage');
const gpayBtn = document.getElementById('gpayBtn');
const rupayBtn = document.getElementById('rupayBtn');
const internetBankingBtn = document.getElementById('internetBankingBtn');
onlinePaymentBtn.addEventListener('click', () => {
 onlineOptions.classList.remove('hidden');
 cashMessage.classList.add('hidden');
});
cashPaymentBtn.addEventListener('click', () => {
 cashMessage.classList.remove('hidden');
 onlineOptions.classList.add('hidden');
});
gpayBtn.addEventListener('click', () => {
 const upiUrl =
"upi://pay?pa=smartdine@upi&pn=SmartDine&tn=OrderPayment&am=100&cu=INR";
 window.location.href = upiUrl;
});
rupayBtn.addEventListener('click', () => {
 alert("Redirecting to Rupay payment gateway (demo).");
});
internetBankingBtn.addEventListener('click', () => {
 alert("Redirecting to Internet Banking portal (demo).");
});

```

menuitems entities

```

{
 "name": "MenuItem", "type": "object", "properties": {
 "name": { "type": "string", "description": "Name of the dish"
 }, "description": { "type": "string", "description": "Description of the dish"
 }, "price": { "type": "number", "description": "Price in INR" }, "category_id": {
 "type": "string", "description": "ID of the category this item belongs to" }, "category_name": {
 "type": "string", "description": "Name of the category for easy display" }, "image_url": {
 "type": "string", "description": "URL to dish image" }, "is_special": {
 "type": "boolean", "default": false, "description": "Is this a special item for the homepage?"
 }, "is_available": { "type": "boolean", "default": true, "description": "Is this item currently available?"
 }, "dietary_tags": { "type": "array", "items": { "type": "string", "enum": ["veg", "non-veg", "jain", "spicy"]
 }, "description": "Dietary information" } }, "required": ["name", "price", "category_id", "category_name"], "rsl": {
 "read": {}, "write": { "user_condition": { "role": "admin" } } } } order entities
{
 "name": "Order", "type": "object", "properties": {
 "customer_name": { "type": "string", "description": "Customer name" }, "customer_email": {
 "type": "string", "description": "Customer email" }, "table_number": { "type": "string", "description":
 "Table number"
 }, "items": {

```

```

"type": "array", "items": { "type": "object", "properties": {
 "menu_item_id": { "type": "string"}, "name": { "type": "string"}, "price": { "type": "number" },
 "quantity": { "type": "number" } }, "description": "Ordered items" }, "total": {
"type": "number", "description": "Total order amount in INR" }, "status": {
"type": "string", "enum": ["pending", "confirmed", "preparing", "ready", "served", "completed"],
"default": "pending", "description": "Order status" }, "special_instructions": { "type": "string",
"description": "Special instructions" } }, "required": ["customer_email", "items", "total"]
} category entities { "name": "Category", "type": "object", "properties": { "name": { "type": "string",
"description": "Name of the menu category"}, "description": { "type": "string", "description": "Optional
description for the category"
}
}, "required": ["name"], "rls": { "read": {}, "write": { "user_condition": { "role": "admin"
} } } }
}

```

layout.js

```

import React from "react";
import { Link, useLocation } from "react-router-dom";
import { createPageUrl } from "@/utils";
import { base44 } from "@api/base44Client";
export default function Layout({ children, currentPageName }) {
 const location = useLocation();
 const [mobileMenuOpen, setMobileMenuOpen] = React.useState(false);
 const [user, setUser] = React.useState(null);
 React.useEffect(() => {
 const fetchUser = async () => {
 try {
 const currentUser = await base44.auth.me();
 setUser(currentUser);
 } catch (error) {
 console.log("Not logged in");
 }
 };
 fetchUser();
 }, []);
 const navItems = [
 { name: "Home", path: createPageUrl("Home"), showToAll: true }, { name: "Menu", path:
createPageUrl("Menu"), showToAll: true }, { name: "Premium", path: createPageUrl("Premium"),
showToAll: true }, { name: "Contact", path: createPageUrl("Contact"), showToAll: true }, { name:
"Admin", path: createPageUrl("Admin"), showToAll: false, adminOnly: true }, { name: "Cart", path:
createPageUrl("Cart"), showToAll: true },];
 const visibleNavItems = navItems.filter(item =>
 item.showToAll || (item.adminOnly && user?.role === 'admin')
);
}

```

```

return (
 <div style={{ minHeight: '100vh' }}>
 <style>`
 @media (max-width: 768px) {
 .desktop-nav {
 display: none !important;
 }
 .mobile-menu-btn {
 display: block !important;
 }
 }
 @media (min-width: 769px) {
 .desktop-nav {
 display: flex !important;
 }
 .mobile-menu-btn {
 display: none !important;
 }
 }
 `</style>
 <header style={{
 background: '#e63946', color: 'white', padding: '15px 30px', display: 'flex', justifyContent: 'space-between',
 alignItems: 'center', position: 'sticky', top: 0, zIndex: 1000 }}>
 /* Logo */
 <Link to={createPageUrl("Home")} style={{ textDecoration: 'none' }}>
 <div style={{ fontSize: '24px', fontWeight: 'bold', color: 'white' }}>
 SmartDine
 </div>
 </Link>
 <nav className="desktop-nav" style={{ display: 'flex', gap: '15px' }}>
 {visibleNavItems.map((item) => {
 const isActive = location.pathname === item.path;
 return (
 <Link
 key={item.name}
 to={item.path}
 style={{
 background: isActive ? '#faa307' : '#ffba08', color: isActive ? 'white' : '#333', textDecoration:
 'none', fontWeight: 'bold', padding: '8px 16px', borderRadius: '6px', transition: 'background 0.3s ease'
 }}
 >

```

```

 {item.name}
 </Link>
);
 }}
</nav>
<button
 className="mobile-menu-btn"
 onClick={() => setMobileMenuOpen(!mobileMenuOpen)}
 style={{
 display: 'none', fontSize: '28px', background: 'none', border: 'none', color: 'white', cursor:
'pointer'
 }}

 {mobileMenuOpen ? '×' : '≡'}
</button>
</header>
{mobileMenuOpen && (
 <div style={{
 background: '#e63946', padding: '15px', position: 'absolute', top: '60px', right: '20px'
borderRadius: '8px', boxShadow: '0 4px 8px rgba(0,0,0,0.2)', zIndex: 999, display: 'flex',
flexDirection: 'column', gap: '10px'}}>
 {visibleNavItems.map((item) => {
 const isActive = location.pathname === item.path;
 return (
 <Link
 key={item.name}
 to={item.path}
 onClick={() => setMobileMenuOpen(false)}
 style={{
 background: isActive ? '#faa307' : '#ffba08', color: isActive ? 'white' : '#333', textDecoration:
'none', fontWeight: 'bold', padding: '8px 16px', borderRadius: '6px', textAlign: 'center' }}>
 {item.name}
 </Link>);
)
 }}
 </div>
)
<main>{children}
</main>
</div>
);
}

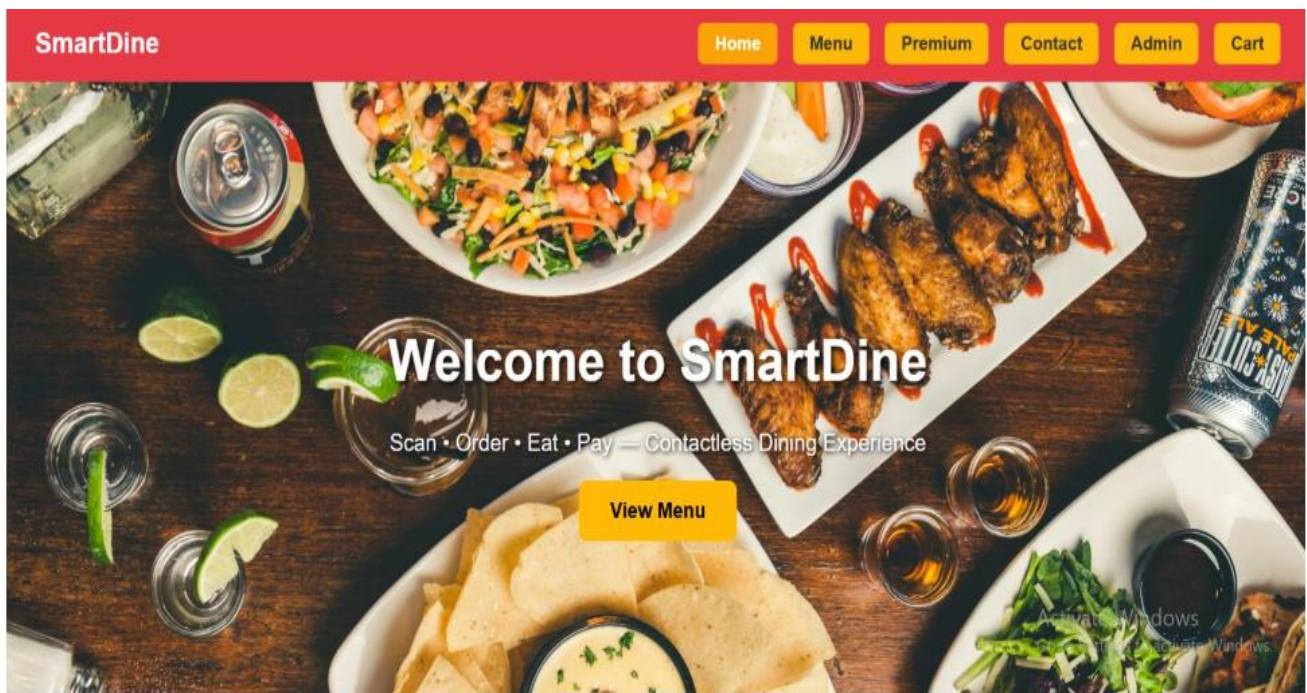
```

# **CHAPTER 4**

## **SAMPLE OUTPUT**

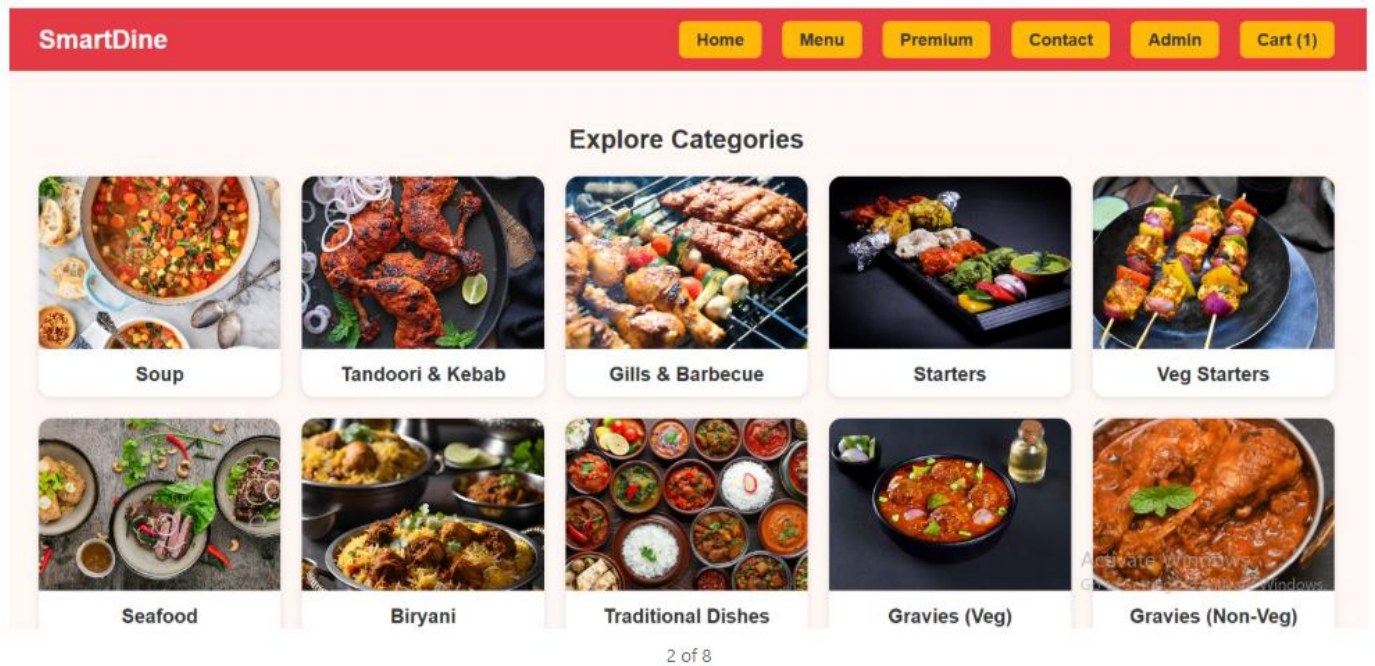
## SAMPLE OUTPUT

### Home Page:

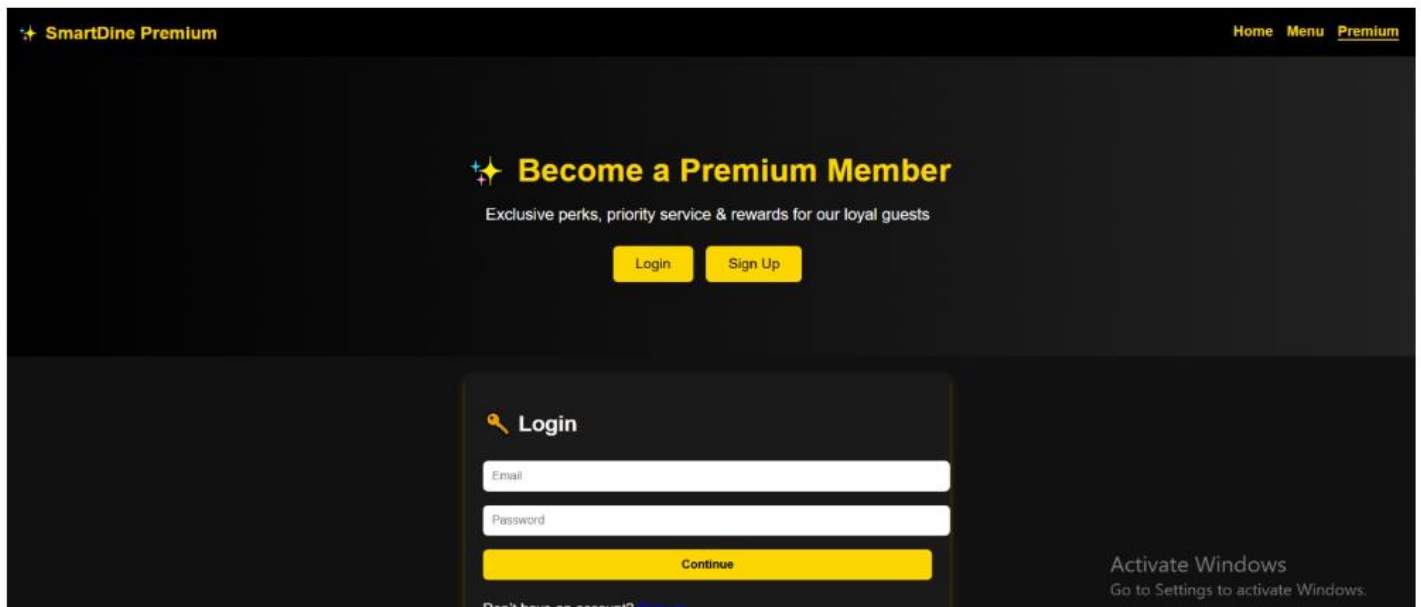


### Menu page:





## Premium page:



## Contact page:



## Get in Touch

We're here to help — whether it's a quick query, reservation issue, or just to say hi!

### Hours of Operation

Mon – Sun: 10:00 AM – 11:00 PM

Excluding Public Holidays

### Call Us

[+91 97899 57524](tel:+919789957524)

[+91 73585 70267](tel:+917358570267)

### Email Us

[nazeefasuman@gmail.com](mailto:nazeefasuman@gmail.com)

[mubash322@gmail.com](mailto:mubash322@gmail.com)

### Table Reservation

Reserve your table in advance for a hassle-free dining experience

Activate Windows

Go to Settings to activate Windows.

# Admin panel Page:

SmartDine

Home

Menu

Premium

Contact

Admin

Admin Panel

Categories

Specials

Orders

Manage Categories

Category name

Add

Current Categories

Biryani

Tandoori

Starters

Desserts

Activate Windows

Go to Settings to activate Windows.

5 of 8

# Cart page:

Your Cart

Review your order and checkout

Customer Details

Name \*

Your name

Email \*

your@email.com

Table Number \*

e.g. A5

Special Instructions

Any dietary restrictions or preferences...

Order Summary

Biryani

-

1

+

₹ 250.00

Paneer Tikka

-

1

+

₹ 180.00

Subtotal

₹ 430.00

GST (5%)

₹ 21.50

To Pay

₹ 451.50

Place Order

Activate Windows

Go to Settings to

## Database for orders:

### Order

...

+ Add

| customer_name   | customer_email         | table_number | items                         | total              | status  | special_instructions | Actions                                                                             |
|-----------------|------------------------|--------------|-------------------------------|--------------------|---------|----------------------|-------------------------------------------------------------------------------------|
| nazeefa         | nazeefasuman@gmail.com | A7           | [{"menu_item_id":"68f2448b... | 385                | pending | WITH LESS SALT       |  |
| nazeefa         | nazeefasuman@gmail.com | A7           | [{"menu_item_id":"68f237c6... | 10.4895            | pending | more sprinkles       |  |
| Nazeefa Suman.K | nazeefasuman@gmail.com | A8           | [{"menu_item_id":"68f23a5f... | 73.458             | pending | Noo                  |  |
| Name            | nazeefasuman@gmail.com | A8           | [{"menu_item_id":"68f237c6... | 56.678999999999995 | pending | No                   |  |

## Menu items:

| name               | description                       | price | category_id             | category_name         | image_url                     | is_special | is_availab |
|--------------------|-----------------------------------|-------|-------------------------|-----------------------|-------------------------------|------------|------------|
| Paneer Tikka       | Cubes of paneer marinated ...     | 350   | CATEGORY_ID_STARTERS... | Starters (Veg)        | https://images.unsplash.co... | true       | true       |
| Chicken 65         | Spicy, deep-fried chicken di...   | 420   | CATEGORY_ID_STARTERS... | Starters (Non-Veg)    | https://images.unsplash.co... | true       | true       |
| Palak Paneer       | Paneer in a thick paste mad...    | 450   | CATEGORY_ID_MAIN_VEG    | Main Course (Veg)     | null                          |            | true       |
| Butter Chicken     | Chicken in a mildly spiced t...   | 550   | CATEGORY_ID_MAIN_NON... | Main Course (Non-Veg) | https://images.unsplash.co... |            | true       |
| Garlic Naan        | Soft Indian bread with garli...   | 90    | CATEGORY_ID_BREADS      | Breads                | null                          |            | true       |
| Hyderabadi Biryani | A flavorful rice dish with spi... | 650   | CATEGORY_ID_RICE        | Rice & Biryani        | https://images.unsplash.co... |            | true       |
| Gulab Jamun        | Soft, melt-in-your-mouth, fri...  | 150   | CATEGORY_ID_DESSERTS    | Desserts              | null                          |            | true       |
| Paneer Tikka       | Cottage cheese marinated i...     | 180   | REPLACE_WITH_STARTER... | Starters              | null                          | true       | true       |
| Chicken 65         | Spicy deep-fried chicken ap...    | 220   | REPLACE_WITH_STARTER... | Starters              | null                          | true       | true       |
| Butter Chicken     | Creamy tomato-based curr...       | 280   | REPLACE_WITH_MAIN_ID    | Main Course           | null                          | true       | true       |
| Palak Paneer       | Cottage cheese in spinach ...     | 220   | REPLACE_WITH_MAIN_ID    | Main Course           | null                          |            | true       |

## Orders:

| customer_name   | customer_email            | table_number | items                          | total              | status  | special_instructions | Actions                                                                               |
|-----------------|---------------------------|--------------|--------------------------------|--------------------|---------|----------------------|---------------------------------------------------------------------------------------|
| nazeefa         | nazeefasuman@gmail.com    | A7           | [{"menu_item_id":"68f2448b...  | 385                | pending | WITH LESS SALT       |  |
| nazeefa         | nazeefasuman@gmail.com    | A7           | [{"menu_item_id":"68f237c68... | 10.4895            | pending | more sprinkles       |  |
| Nazeefa Suman.K | nazeefasuman@gmail.com    | A8           | [{"menu_item_id":"68f23a5f7... | 73.458             | pending | Noo                  |  |
| Name            | nazeefasuman@gmail.com    | A8           | [{"menu_item_id":"68f237c68... | 56.678999999999995 | pending | No                   |  |
| Irshad          | irshadahmed7863@gmail.com | A10          | [{"menu_item_id":"68f2448b...  | 808.5              | pending | Extra cheese         |  |

# **CHAPTER-6**

## **TESTING**

## **6.1 TESTING:**

Testing is a crucial phase in the software development life cycle that ensures the developed system meets its requirements and functions correctly. In the Smart Dine project, testing was performed to verify that all modules — such as table management, menu display, order processing, and billing — work seamlessly and deliver a smooth user experience.

The main goal of testing is to identify and fix any errors or inconsistencies in the system before deployment, ensuring a reliable and efficient restaurant management solution.

### **6.2 Objectives of Testing.**

- ◆ To verify that the system meets the specified requirements.
- ◆ To detect and correct errors, bugs, and inconsistencies.
- ◆ To confirm that the system is user-friendly and performs efficiently under load.
- ◆ To validate the integration of different modules such as QR scanning, order management, and payment processing.

## **6.3 Types of Testing Performed**

### **6.3.1 Unit Testing**

Unit testing focuses on testing individual modules or components of the system. Each module, such as the Menu, Order, and Table modules, was tested independently to ensure it performs its functions correctly.

#### **Example:**

Testing the QR Code scanner to ensure it fetches the correct table details.

Verifying that the “Add to Order” button correctly adds selected menu items.

### **6.3.2 Integration Testing**

After unit testing, integration testing was performed to ensure that all modules work together properly.

The integration between QR scanning, order placement, and database updates was checked to confirm data consistency.

#### **Example:**

Checking if the order details are correctly linked to the scanned table ID.

Verifying that once the waiter accepts the order, the status updates in both admin and user panels.

### **6.3.3 System Testing**

System testing involved testing the complete Smart Dine application as a whole. It validated the system's performance, reliability, and compliance with specified requirements

#### **Example Tests:**

Checking whether multiple users can place orders simultaneously.

Ensuring that the admin can view all active tables and orders in real time.

### **6.3.4 User Acceptance Testing (UAT)**

This testing phase involved real users (restaurant staff and customers) who interacted with the Smart Dine system. Their feedback was collected to evaluate usability, design clarity, and ease of operation.

#### **Feedback:**

The interface was easy to use and responsive.

The QR scanning and order tracking features were well appreciated.

Minor improvements were suggested for faster order updates.

## **6.5 Results and Findings**

The testing results show that all functional modules performed correctly with no critical issues. Minor user interface adjustments were made to improve responsiveness on mobile devices. The system successfully handled multiple user sessions, confirming scalability and reliability.

# **CHAPTER-7**

## **CONCLUSION**



## **7.1 CONCLUSION**

Testing confirmed that the Smart Dine system is functional, stable, and user-friendly. All modules — from QR scanning to order processing and admin control — operated seamlessly after debugging. The application is ready for deployment in a real restaurant environment.

## **Bibliography**

- ✓ **[www.ninjahub.com](http://www.ninjahub.com)**
- ✓ **[www.geeksforgeeks.com](http://www.geeksforgeeks.com)**
- ✓ **[www.stacksssoverflow.com](http://www.stacksssoverflow.com)**